Cálculo Computacional e Estatística

Organização de dados e criação de conjuntos de dados – Coleta dados de máquina (PI)

Antes fazer qualquer análise de dados, vamos coletar dados de interesse e que estejam integrados a aplicação do projeto de PI.
O PI tem por objetivo monitoramento de no mínimo CPU, memória e Hard Disk.

Usaremos estes dados para criar nossos conjuntos e assim aplicar as técnicas da disciplina de cálculo.

Para capturar dados de máquina, o python usa a biblioteca psutil.

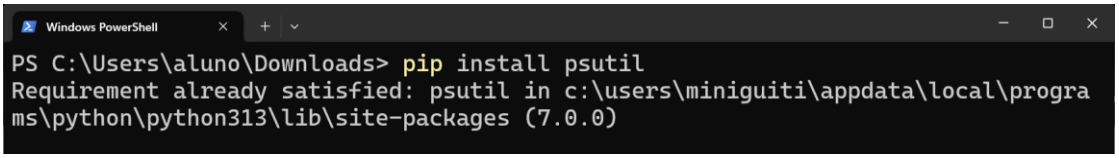
A **psutil** (sistema python e utilitários de processo) é uma biblioteca de **plataforma cruzada** para recuperar informações sobre processos em execução e utilização do sistema (CPU, memória, discos,rede, sensores) em Python . É útil principalmente para monitoramento de sistema, criação de perfil, limitação de recursos de processo e gerenciamento de processos em execução.

Mas o que é plataforma cruzada?

- **Aplicação Nativa:** Criada especificamente para cada sistema (ex.: Android; IOs)
- **Aplicação em plataforma cruzada:** Surgiu da utilização de plataformas e linguagens WEB para o desenvolvimento Mobile (para Android, IOs, Windows Phone). É considerada como uma única versão que roda para todos os sistemas.

Configuração do Ambiente:

- **Acesse:** https://pypi.org/project/psutil/#files para diferentes versões
- No PowerShell (Windows) ou terminal (Ubuntu), execute `pip install psutil`
- **Atenção (Windows)**: caso o `pip` não seja reconhecimento, verifique a configuração da variável de ambiente do Python no Path



Captura de Dados de Máquina (CPU)

Agora vamos entender como e o que capturar de dados de máquina:

CPU(cpu_times):

- Faça import da lib: **`import psutil`**
- Função para captura: **`psutil.cpu_times(percpu = False)`**
- Retorno: **`scputimes(user=9854.578125,system=4752.328124999996, idle=27844.5, interrupt=227.703125, dpc=816.546875)`**

Retorna os tempos de CPU do sistema como uma tupla nomeada. Cada atributo representa os segundos que a CPU gastou no modo determinado. A disponibilidade dos atributos varia de acordo com a plataforma:

- **scputimes:** estatística de tempo de cpu
- **cpu user:** tempo gasto pelo usuário
- **system:** tempo gasto pelo sistema
- **idle:** tempo de cpu ociosa
- **dpc** (defereed procedures call – chamadas de procedimento adiada) - é a porcentagem de tempo que o processador gasta recebendo e atendendo chamadas de procedimento adiado. São chamadas ao processador que não responde ao processamento solicitado.

Veja que para várias solicitações da cpu_times os valores vão mudando. Podemos então criar uma tabela com essas informações.

CPU(cpu_percent):

- Função para captura: **`psutil.cpu_percent(interval = None, percpu = False)`**

Retorne um float que representa a utilização atual da CPU de todo o sistema como uma porcentagem. Quando o intervalo é > 0.0 compara os tempos de CPU do sistema decorridos antes e depois do intervalo (bloqueio). Quando percpu é True retorna uma lista de flutuações que representam a utilização como uma porcentagem para cada CPU. O primeiro elemento da lista se refere à primeira CPU, o segundo elemento à segunda CPU e assim por diante. A ordem da lista é consistente nas chamadas.

Exemplo:

```
>>> psutil.cpu_percent(interval=1)
29.4
>>> psutil.cpu_percent(interval=None)
36.3
>>> psutil.cpu_percent(interval=1, percpu=True)
[40.3, 26.2, 33.8, 23.1]
```

CPU(cpu_count):

- Função para captura: **psutil.cpu_count()**
- Função para captura com paramêtro: **psutil.cpu_count(logical=False)**

Retorne o número de CPUs lógicas no sistema (o mesmo que os cpu_count no Python 3.4) ou none se indeterminado. Núcleos lógicos significa o número de núcleos físicos multiplicado pelo número de threads que podem ser executados em cada núcleo (isso é conhecido como Hyper Threading). Se lógicas é false retorna o número de núcleos físicos apenas

Exemplos:

```
>>> psutil.cpu_freq()
scpufreq(current=2511.0, min=0.0, max=2712.0)

>>> psutil.cpu_freq(percpu=True)
```

Retorna a frequência da CPU como um nome duplo, incluindo as frequências atual, mínima e máxima expressas em Mhz.

Capturando outros dados:

Façam a parte de Disco e Memória. Veja as dicas a seguir:

```
>>> psutil.disk_partitions()
[sdiskpart(device='C:\\', mountpoint='C:\\', fstype='NTFS',
opts='rw,fixed'), sdiskpart(device='D:\\', mountpoint='D:\\',
fstype='NTFS', opts='rw,fixed')]

>>> psutil.disk_usage('C:\\')
 sdiskusage(total=261480951808, used=229084987392, free=32395964416,
percent=87.6
```

psutil.disk_usage(caminho) - Retorna estatísticas de uso de disco sobre a partição que contém o caminho dado como uma tupla nomeada incluindo espaço total , usado e livre expresso em bytes, mais a porcentagem de uso.

Faça o teste das seguintes capturas:

```
>>> psutil. disk_partitions (all = False)
>>> psutil.virtual_ memory()
```